



find successor
of x

Binary Search Tree

Insertion :

- Causes the dynamic set to be modified
- The tree needs to be modified s.t. the BST property continues to hold

Input : a BST node z for which $z.key$ has been set, and
 $z.left = NIL$
 $z.right = NIL$

Tree-Insert (T, z)

$x = T.root$ // Node being compared with z
 $y = NIL$ // y will be the parent of z
while $x \neq NIL$

$y = x$
 if $z.key < x.key$
 $x = x.left$
 else $x = x.right$

$z.p = y$ // found the location - insert z with parent y

if $y == NIL$

$T.root = z$ // Tree was empty

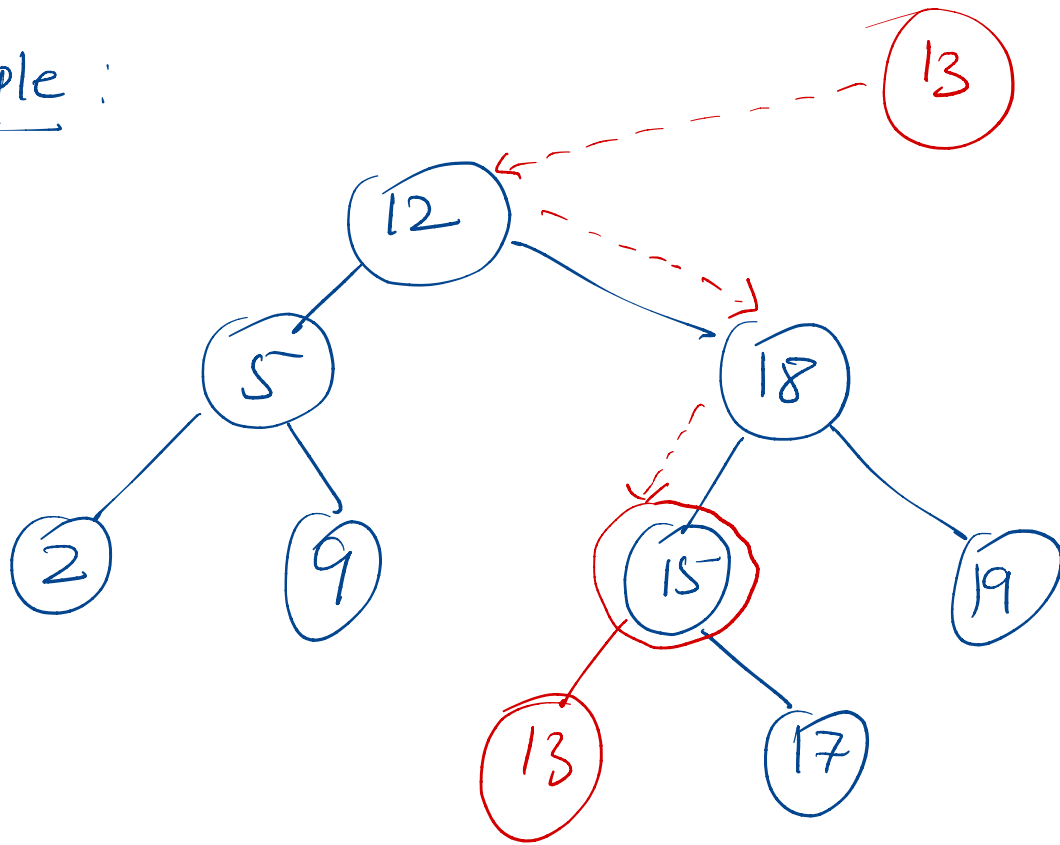
else if $z.key < y.key$

$y.left = z$

else $y.right = z$

$O(h)$
complexity

Example :



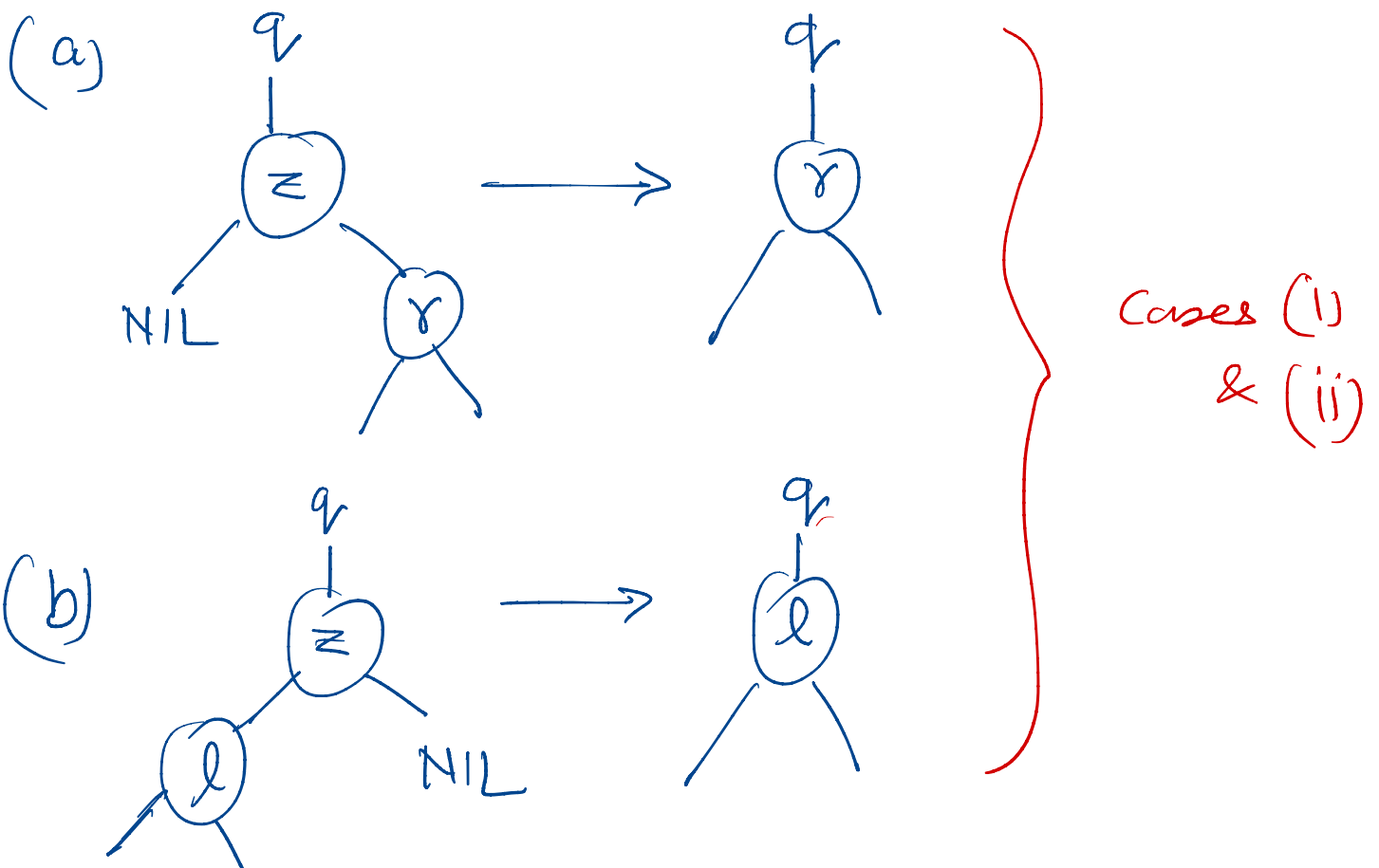
Deletion : — Delete a node $\neq$ from
BST T

— 3 basic cases :

- (i) If z has no children then simply remove it by modifying its parent to replace z with NIL as child.
- (ii) If z has just 1 child, then elevate that child to take z 's position in the tree by modifying z 's parent to replace z by z 's child

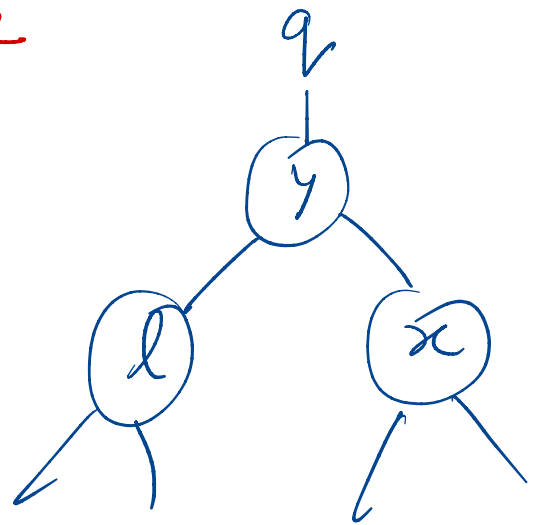
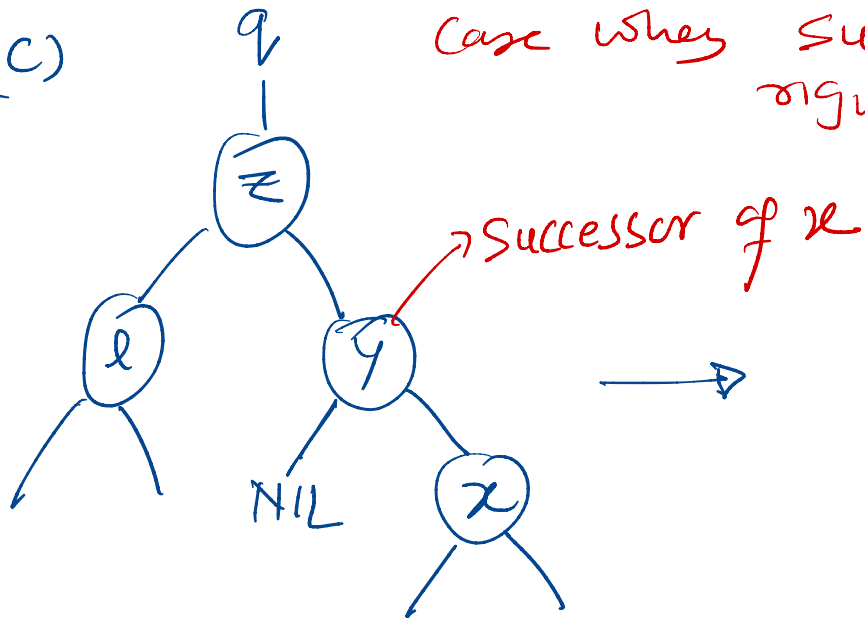
(iii) If z has 2 children, find z 's successor y - which must belong to z 's right subtree - and move y to take z 's position in the tree.

The rest of z 's original right subtree becomes y 's new right subtree & z 's left subtree becomes y 's new left subtree

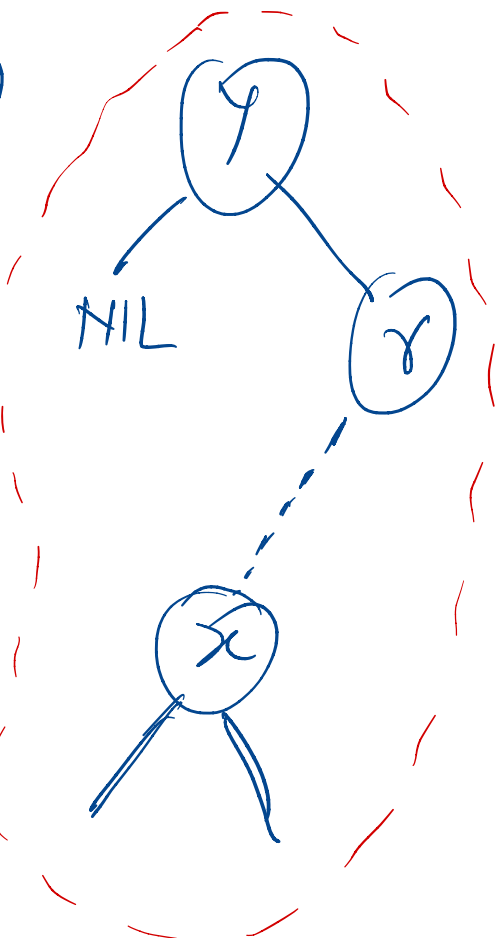
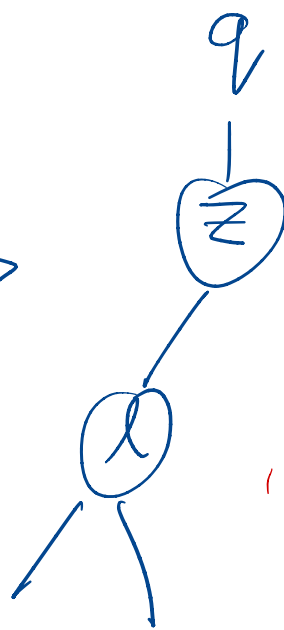
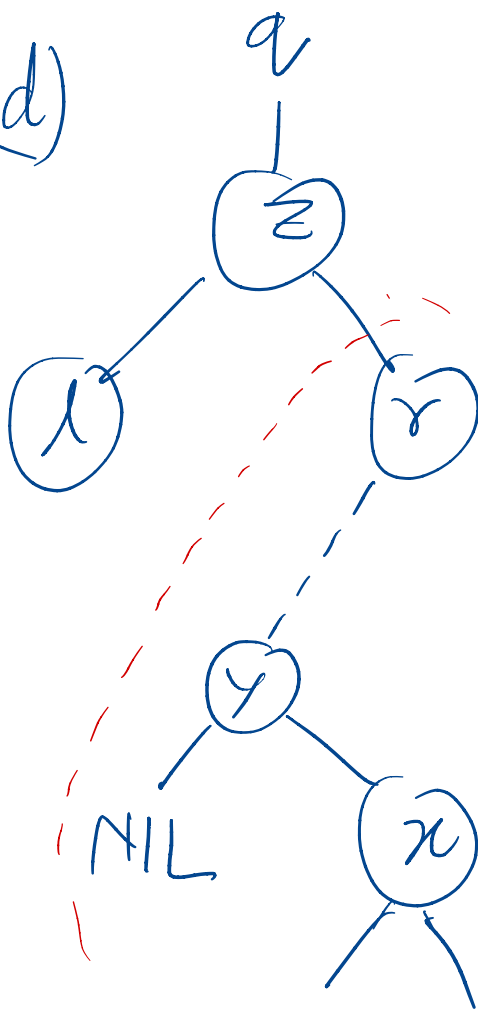


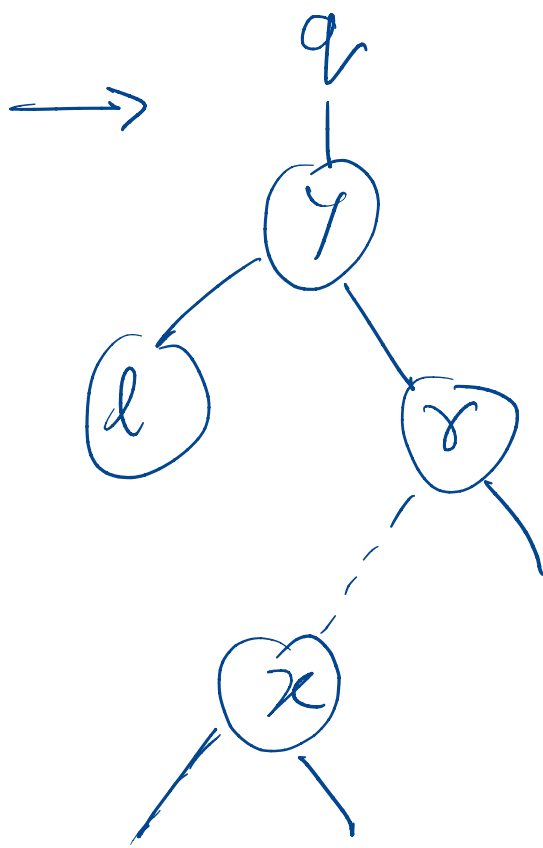
(c)

Case when Successor y is the right child of z



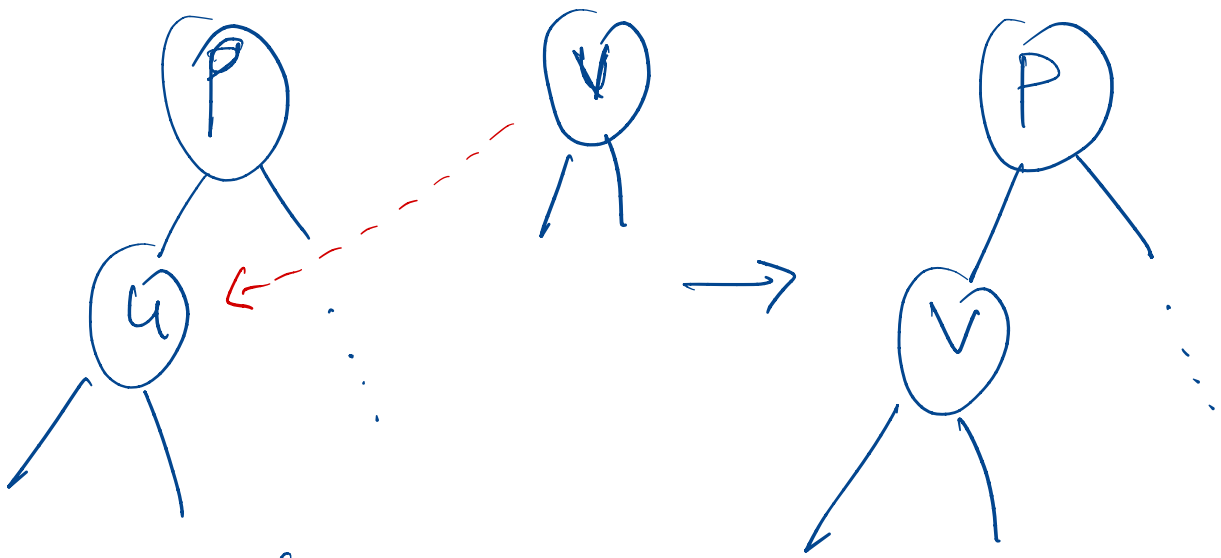
(d)





Transplant operation

- ↳ replaces one subtree as a child of its parent with another subtree
- ↳ Define Transplant that replaces subtree rooted at node u with subtree rooted at node v . Set Parent Pointers appropriately



(When u is left of P . Symmetric case when u is right child)

Transplant (T, u, v)

if $u.p == \text{NIL}$

$T.\text{root} = v$

else if $u == u.p.\text{left}$

$u.p.\text{left} = v$

else $u.p.\text{right} = v$

if $v \neq \text{NIL}$

$v.p = u.p$

// Set the Parent

- Define Tree-Delete using Transplant

Tree-Delete (T, z)

if $z.left == NIL$ replace z by its right child
 Transplant ($T, z, z.right$)

else if $z.right == NIL$ replace z by its left child
 Transplant ($T, z, z.left$)

else successor of z
 $y = \text{Tree_minimum}(z.right)$
 if $y \neq z.right$

replace y by its right child ← Transplant ($T, y, y.right$)
 $y.right = z.right$
 $y.right.p = y$ } z 's right child becomes y 's right child

Transplant (T, z, y) $\rightarrow$ replace z
by successor y

$$y.\text{left} = z.\text{left}$$

$$y.\text{left}.p = y$$

} set y 's left
to z 's left

